

Machine Learning and Agentic AI for Macroeconomic Research

10: Case Studies — Agentic AI in Economic Research

Zhigang Feng

Spring 2026

Lecture Agenda

Part I: Foundations

1. Why case studies?
2. The research workflow arc
3. Connecting Lec09 concepts

Part II: Case Study 1

Building a Paper Review Skill

4. The problem & naive baseline
5. Skill architecture
6. Prompt engineering
7. Running and output

Part III: Case Study 2

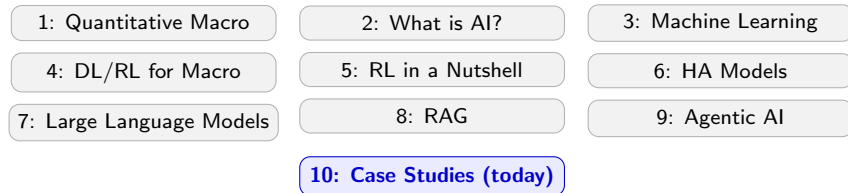
MEPS Uninsurance Analysis

8. The problem & naive baseline
9. Data architecture
10. Visualization and insights

Part IV: Synthesis

11. Lessons and patterns
12. Lab: MEPS data analysis

Where We Are in the Course



The progression:

- Lectures 1–6: Understanding models and AI/ML methods for solving them
- Lectures 7–8: Understanding the language model revolution (LLM + RAG)
- Lecture 9: Understanding agentic AI tools (skills, agents, sub-agents)
- **Lecture 10: Putting it all together** — two complete research workflows

Why Case Studies?

The Gap Between Tools and Research

Lecture 9 gave you the vocabulary: skills, agents, sub-agents, CLAUDE.md, harness engineering, the 5-pillar workflow.

But knowing the tools $\neq$ knowing how to use them for research.

The hard part is *design*:

- How do you decompose a research task into agent-sized steps?
- How do you encode domain knowledge into prompts?
- How do you validate AI-generated output?
- When does the agent save time vs. create new problems?

Today: two complete case studies that answer these questions.

1. **Case Study 1:** Automating academic paper review (NLP + domain expertise)
2. **Case Study 2:** Assembling a 25-year panel from MEPS data (data + visualization)

The Research Workflow Arc

Both case studies follow the same five-stage arc, adapted from Goldsmith-Pinkham (2026):



Stage	Case Study 1 (Paper Review)	Case Study 2 (MEPS Data)
Problem	Reviewing papers is hard	24 years of survey data
Infrastructure	Skill folder + YAML config	Claude Code + Python env
Architecture	7-phase review pipeline	Download → Harmonize → Analyze
Capability	Prompt engineering + filters	Weighted statistics by group
Insights	Structured referee report	6 publication-quality figures

Reference: Paul Goldsmith-Pinkham, “Large Datasets and Structured Databases: Claude Code for Economists,” 2026.

What Lec09 Concepts Recur Today

Lec09 Concept	Case Study 1 (Paper Review)	Case Study 2 (MEPS Data)
Skills	The entire review-paper skill	Ad hoc prompts (no skill needed)
Agents / sub-agents	3 calibration agents, N section reviewers	Single-session workflow
Shared directives	shared-directives.md (tone, confidence)	CLAUDE.md (project context)
Iterative workflow	Edit prompt → rerun → inspect	Prompt → inspect → refine
Pillar 5 (Verify)	Check math claims, judge domain fit	Cross-check vs. Census Bureau

Key observation: Case Study 1 is a *skill* (reusable, multi-phase, quality-gated).

Case Study 2 is *ad hoc* (one-off, 3 prompts, single session).

Both are legitimate. The choice depends on: *How often will you do this? How much quality control do you need?*

Case Study 1: The Problem

The Naive Baseline: Paste a Paper into ChatGPT

The tempting approach: Copy your paper's abstract into ChatGPT and ask "review this paper."

What you get:

- Generic praise: *"The paper makes an important contribution to the literature. . ."*
- Surface-level suggestions: *"The authors should discuss limitations. . ."*
- No verbatim quotes from the paper
- No equation verification
- No domain-calibrated assessment

What's missing:

1. No extraction pipeline — the LLM sees a wall of text, not structured sections
2. No domain calibration — it doesn't know what matters in quantitative macro
3. No specialization — proofs need different attention than methodology
4. No quality gate — hallucinated comments survive unchecked
5. No quote verification — it may "quote" text the paper never said

What a Good Referee Report Looks Like

A structured review has clear components:

Example: refine.ink output format

```
# Regression Discontinuity Design with Distribution

**Date**: 03/03/2026
**Domain**: social_sciences/economics
**Taxonomy**: academic/research_paper

## Overall Feedback
**Estimand definition needs tightening**
The paper defines the estimand as a weighted average
of treatment effects but the weighting function is
never made explicit. In Section 2.3, equation (7)...

## Detailed Comments (20)
### 1. Sign error in Theorem 2 proof
**Quote**: > "By dominated convergence,  $\lim_{h \rightarrow 0} \dots$ "
**Feedback**: The dominated convergence step requires...
```

The Paper We Will Review

Demo Paper

Markov Equilibria with Quasi-Hyperbolic Discounting

Zhigang Feng (University of Nebraska Omaha) and Manuel S. Santos (University of Miami)

Content:

- Neoclassical growth model with (β, δ) quasi-hyperbolic preferences
- Proves existence of Markov equilibria via a sequence of ε -equilibria
- Establishes saddle-path stability of steady states (ruling out indeterminacy)
- Numerical computation of equilibrium policy functions

Why this paper is a good test case:

1. **Proofs** that need step-by-step verification (Theorems on existence, stability)
2. **Numerical results** that can be checked for internal consistency
3. **Domain-specific methodology** (dynamic programming, Markov equilibrium)
4. **Cross-section claims** (discussion section vs. formal results)

What We Want the Skill to Produce

Given the paper as input, the skill should output:

Overview Assessment

- 4–8 macro-level issues
- Each with section/equation references
- Concrete remediation suggestions
- Recommendation: accept / minor / major / reject
- Key revision targets

Detailed Comments

- 8–25 section-level comments
- Each anchored to a **verbatim quote**
- Severity: critical / major / minor
- Confidence: high / medium / low
- Feedback: 3–8 sentences with concrete fix

The design challenge: How do we break this into stages that are each small enough for one agent call, but together cover the full review?

⇒ This is an **orchestration problem** — exactly what Claude Code skills are designed for.

The Design Challenge: Why Decomposition?

Why can't a single prompt do this?

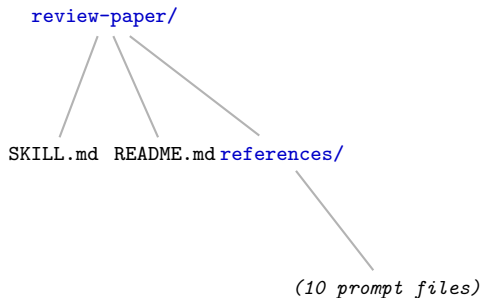
1. **Context window limits.** A 60-page paper + all review instructions + output format exceeds a single agent call. The skill loads prompts on demand.
2. **Specialization.** Checking a proof step-by-step requires different attention than evaluating whether a methodology section properly identifies its assumptions. Different prompts for different section types.
3. **Quality control.** Without an editorial filter, you get 40 comments — half generic, half wrong. A dedicated quality gate catches hallucinated quotes and generic advice.

The analogy: You would not ask a single RA to simultaneously extract data, calibrate review criteria, write deep section reviews, and edit the final report. You decompose the task and assign specialized roles.

⇒ The skill decomposes the review into **7 sequential phases**, with parallelism *within* phases.

Case Study 1: Skill Architecture

The File Layout (1/2)



Two types of files:

1. The orchestrator (`SKILL.md`)

- Defines the 7-phase pipeline
- Tells Claude *which* reference files to read and *when*
- Written in plain English

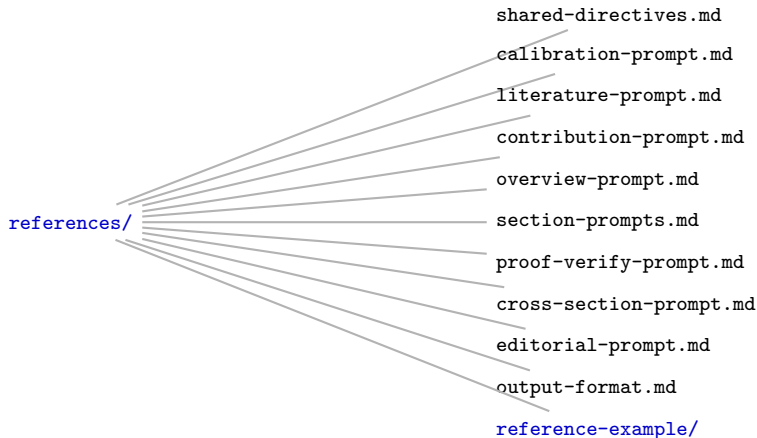
2. Reference prompts (`references/*.md`)

- Specialized instructions for each phase
- Loaded on demand (not all at once)
- Each file is self-contained

Key insight: The skill is a *folder*, not a single file. Installation is copying a folder.

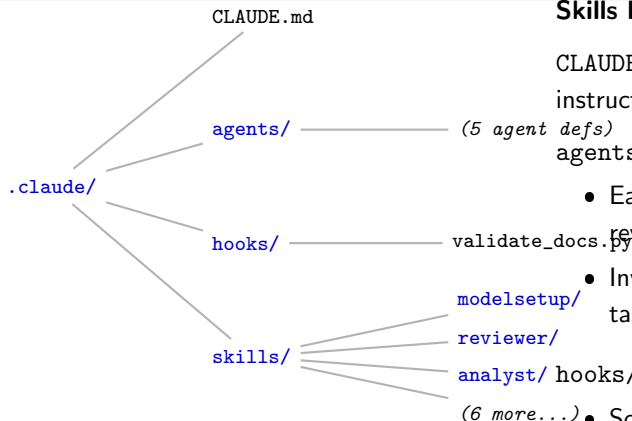
The File Layout (2/2)

Inside references/:



Each file contains **specialized instructions** loaded on demand — one phase at a time.

Alternative: The .claude/ Project Folder



Skills live inside .claude/:

`CLAUDE.md` — persistent project memory & global instructions for Claude

`agents/` — subagent persona definitions

- Each agent has a specialized role (algorithmist, reviewer, ...)

- Invoked by the orchestrator during multi-agent tasks

`hooks/` — event-driven automation

- Scripts triggered by Claude Code events

- E.g., validate documentation after each edit

`skills/` — multiple skill folders, same structure as before

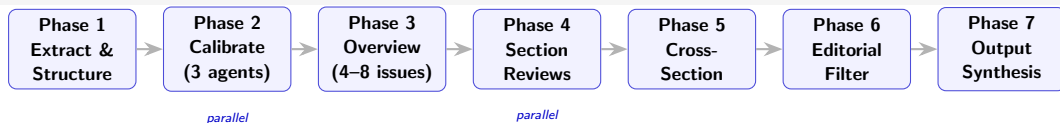
The YAML Frontmatter

Every skill starts with YAML frontmatter in SKILL.md:

```
---
name: review-paper
description: >
  Review an academic paper using the coarse multi-stage
  peer review pipeline. Produces a refine.ink-format
  markdown report with macro-level issues and detailed
  comments with verbatim quotes. Use when the user wants
  to review a paper, get feedback on a manuscript, or
  generate a referee report.
  Triggers on: "review this paper", "peer review",
  "referee report", "/review-paper path/to/paper.pdf".
---
```

- name: Used for routing — /review-paper invokes this skill
- description: Claude pattern-matches these phrases to decide *when* to invoke the skill automatically
- Arguments: \$ARGUMENTS = the path to the paper file (PDF, TeX, etc.)

The 7-Phase Pipeline



Phase	Name	What It Does
1	Extract & Structure	Read paper, parse into sections, classify types, extract claims
2	Calibrate	3 parallel agents: domain calibration, literature search, contribution extraction
3	Overview	Generate 4–8 macro-level issues + recommendation
4	Section Reviews	Parallel agents with specialized prompts (proof, methodology, results, ...)
5	Cross-Section	Check: do discussion claims match formal results?
6	Editorial Filter	7-step quality gate: dedup, quote verify, severity assign, humanize
7	Output Synthesis	Render final review in refine.ink format

Why 7 Phases, Not 1?

Naive approach: “Read this paper and write a comprehensive review.” (One giant prompt.)

Three reasons for decomposition:

1. Context window

A 60-page paper + all review instructions + output template exceeds a single call.

The skill loads only the prompts needed for each phase.

2. Specialization

Proof-checking requires step-by-step mathematical re-derivation.

Methodology review requires checking identification assumptions.

One prompt cannot do both well.

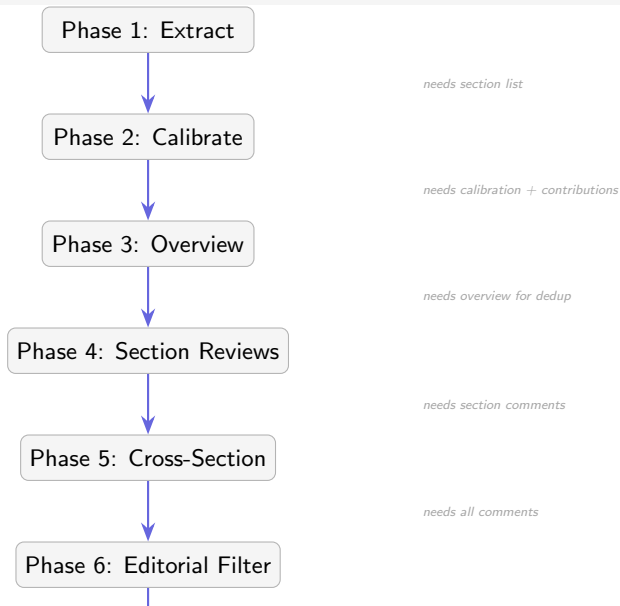
3. Quality control

Without an editorial filter, you get 40 comments — half are generic advice, some “quote” text that doesn’t exist.

The filter catches these.

⇒ **Parallelism is within phases** (3 calibration agents, N section agents), **not between phases** (each phase depends on the previous).

Sequential Dependencies



The Orchestrator Pattern in SKILL.md

The SKILL.md file orchestrates by telling Claude what to read and when. Here is Phase 2:

```
## Phase 2: Calibrate (3 Parallel Agents)
```

```
Launch 3 agents simultaneously using the Agent tool:
```

```
**Agent 1 -- Domain Calibration**
```

- Read: references/calibration-prompt.md
- Read: references/shared-directives.md
- Input: paper metadata + abstract
- Output: domain-specific review criteria

```
**Agent 2 -- Literature Search**
```

- Read: references/literature-prompt.md
- Use WebSearch to find 8-10 related papers
- Output: related work with relevance notes

```
**Agent 3 -- Contribution Extraction**
```

- Read: references/contribution-prompt.md
- Input: full paper text
- Output: stated contributions as bullet list

```
Collect all 3 results before proceeding to Phase 3.
```

Reference Files = Specialized Prompts

Reference File	Phase	Role
shared-directives.md	All	Tone, confidence gate, quote rules, steelman protocol
calibration-prompt.md	2	Generate domain-specific review criteria
literature-prompt.md	2	WebSearch for 8–10 related papers
contribution-prompt.md	2	Extract the paper's stated contributions
overview-prompt.md	3	Identify 4–8 macro-level issues + recommendation
section-prompts.md	4	6 specialized review variants (proof, methodology, results, literature, discussion, general)
proof-verify-prompt.md	4b	Adversarial two-pass proof verification
cross-section-prompt.md	5	Check discussion vs. formal results alignment
editorial-prompt.md	6	7-step quality gate
output-format.md	7	refine.ink rendering template

Key design principle: Each file is self-contained. Only the agents that need a file read it. This keeps each agent's context window efficient.

Why Reference Files Live Outside SKILL.md

Q: Why not put all the prompts directly into SKILL.md?

A: Three reasons.

1. **Context efficiency.** The section reviewer only reads `section-prompts.md` and `shared-directives.md`. It never sees `editorial-prompt.md`. If everything were in one file, every agent would load every prompt.
2. **Modularity.** You can improve the editorial filter without touching the section prompts. Each file is independently editable and testable.
3. **Reusability.** `shared-directives.md` is read by every phase. If it were embedded in SKILL.md, you'd have to duplicate it or use fragile references.

Analogy: A referee coordinator gives each sub-referee a different rubric. The coordinator's job is to assign tasks and collect results — not to memorize all the rubrics.

Case Study 1: Prompt Engineering

Shared Directives: The Foundation

shared-directives.md is injected into *every* review phase. Key excerpts:

Tone

Write as a constructive but direct colleague. Vary phrasing. Do NOT always start with "It would be helpful to..."

Banned Vocabulary

Never use: crucial, comprehensive, robust, multifaceted, delve, landscape, facilitate, holistic, pivotal, leverages

Confidence Gate

Only claim an error if you can support it concretely with a derivation or citation. If uncertain, phrase as a question.

Steelman Protocol

Before claiming a proof step is wrong: (1) state what the authors intended, (2) check their own defense (remarks, footnotes), (3) verify conditions are needed.

⇒ These directives prevent the most common AI review failures: generic praise, AI-sounding language, hallucinated errors, uncharitable reading.

Why Humanization Matters

Without shared directives:

AI-sounding review

*"This paper makes a **crucial** contribution to the **robust** literature on **multifaceted** aspects of time-inconsistent preferences. The authors **delve** into the **landscape** of Markov equilibria. . ."*

With shared directives:

Human-sounding review

"The paper proves existence of Markov equilibria via ϵ -equilibria. The construction circumvents the non-removable discontinuities in the equilibrium mapping, which is the core technical challenge. However, Assumption 3 appears stronger than needed for the stability result in Section 4. . ."

The lesson: If you want AI output that does not read as AI output, you must *explicitly encode* the anti-patterns. Vocabulary bans + tone instructions + the steelman protocol together produce natural,

The Confidence Gate

The single most important prompt engineering pattern in the skill:

Rule: For mathematical claims, show the correct derivation step-by-step. If you cannot support a claim concretely, phrase it as a question.

Why this matters for economics papers:

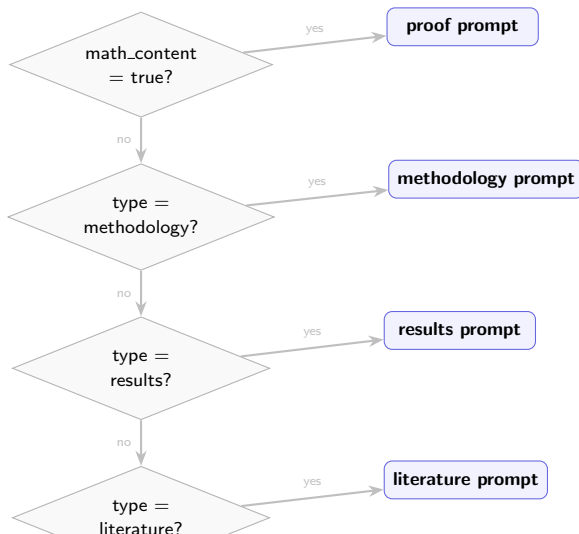
- AI models hallucinate mathematical errors — they confidently assert that a correct proof step is wrong
- The confidence gate forces the model to *show its work* before asserting an error
- If it cannot derive the error, it must say: “Is it possible that...?”

The contribution inversion test (from `shared-directives.md`):

“Before asserting a key result is wrong, check: does the abstract claim the opposite? If the abstract says ‘we prove X’ and you are about to say ‘X is false,’ you have almost certainly made an error. The authors, who spent months on this proof, are more likely correct than a single-pass AI review.”

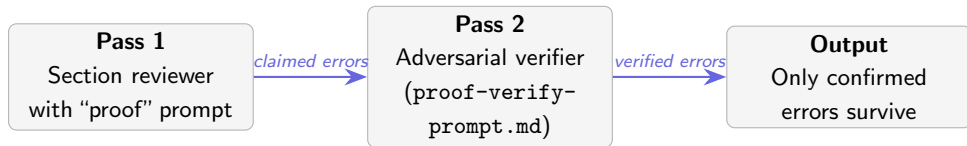
Section Routing: 6 Specialized Prompts

Phase 4 routes each section to the appropriate prompt variant:



The Proof Verification Chain

For sections with mathematical content, the skill uses a **two-pass adversarial architecture**:



- **Pass 1**: Section reviewer finds potential proof issues
- **Pass 2**: Independent verifier re-derives each claimed error from scratch
- Only errors confirmed by *both* passes survive
- If the verifier cannot reproduce the error $\Rightarrow$ the comment is dropped

Analogy: Two referees independently check the math. A claimed error must be confirmed by both before it reaches the author.

The MPE Paper Through the Pipeline: Phase 1

Phase 1: Extract & Structure

The pipeline reads `MPE_Hyper_v44.tex` and produces:

Section	Type	math_content
1. Introduction	introduction	false
2. The Neoclassical Growth Model	methodology	true
3. Existence of Markov Equilibria	proof	true
4. The (β, δ) -Tradeoff	results	true
5. Steady State Stability	proof	true
6. Numerical Simulations	results	false
7. Concluding Remarks	discussion	false
A. Appendix (Proofs)	proof	true

Key claims extracted:

- Theorem 1: Existence of Markov equilibrium via ε -equilibria
- Theorem 2: Saddle-path stability of steady states

The MPE Paper Through the Pipeline: Phases 2–3

Phase 2: Calibrate (3 parallel agents)

Agent	Output
Domain calibration	“Dynamic macro theory, quasi-hyperbolic preferences, Markov equilibrium. Review criteria: check equilibrium definition, verify convergence arguments, assess numerical accuracy.”
Literature search	Harris-Laibson (2001), Krusell-Kuruscu-Smith (2002), Cao-Werning (2018), ...
Contribution extraction	(1) Existence via ε -equilibria, (2) Saddle-path stability, (3) Numerical algorithm

Phase 3: Overview

Generates 4–8 macro-level issues, e.g.:

- **Assumption scope:** Are the concavity conditions in Assumption 2 stronger than needed for the stability result?
- **Numerical validation:** The paper reports accuracy for a single parameterization — how sensitive

The MPE Paper Through the Pipeline: Phase 4

Phase 4: Section-level reviews with specialized prompts

Section	Prompt Variant	Focus
§2 (Growth Model)	methodology	Check model assumptions, identification of equilibrium, completeness of setup
§3 (Existence)	proof + verify chain	Step-by-step verification of Theorem 1; check measurability, convergence
§4 (β, δ -Tradeoff)	results	Numerical consistency, internal coherence of comparative statics
§5 (Stability)	proof + verify chain	Verify saddle-path stability argument, check eigenvalue claims
§6 (Simulations)	results	Code reproducibility, accuracy metrics, sensitivity
§7 (Conclusion)	discussion	Do claims match formal results?
Appendix	proof + verify chain	Verify supporting lemmas

Each section gets its own agent with the appropriate prompt variant. Sections 3, 5, and the Appendix

The Editorial Filter: 7 Steps

Phase 6 applies a 7-step quality gate to all accumulated comments:

1. **Remove low-value comments:** generic advice (“the authors should discuss limitations”), formatting notes, OCR artifacts, restated overview issues
2. **Contradiction check:** Does any comment contradict the paper’s own stated defense? (Check footnotes, remarks)
3. **Quote verification:** Exact character-for-character match against the full paper text. Fuzzy match $> 80\%$ $\Rightarrow$ accept. No match $\Rightarrow$ **drop the comment**
4. **Severity assignment:** critical / major / minor; confidence: high / medium / low
5. **Notation capping:** Max 2–3 purely notational comments (diminishing returns)
6. **Humanize language:** Vary sentence openers, remove banned AI vocabulary, ensure natural cadence
7. **Order by importance:** critical $\rightarrow$ major $\rightarrow$ minor; high confidence first within each tier

$\Rightarrow$ Without this filter, you get 40 comments. After it, you get 12–20 — each substantive, correctly quoted, and ordered by severity.

Quote Verification: The Final Defense

The most embarrassing failure mode: a review comment that “quotes” something the paper never said.

The verification rule:

Exact match $\Rightarrow$ **keep**

Fuzzy match $> 80\%$ $\Rightarrow$ **accept**

No match $\Rightarrow$ **drop**

Quote requirements (from `shared-directives.md`):

- Exact character-for-character copy, including $\text{\LaTeX}$ commands (`\rho`, `\frac`)
- Complete passages — never truncate mid-sentence
- Multi-line equations: include ALL lines
- Minimum 2 full sentences or complete equation block

Safety fallback: If quote verification would drop ALL comments (e.g., because the PDF extraction mangled the text), skip verification and keep originals. This prevents silent failure.

Case Study 1: Running and Output

Installation: Two Lines

```
# Copy the skill folder into your project  
cp -r review-paper-skill/ .claude/commands/review-paper/
```

That's it. A skill is a folder. Installation is copying a folder.

What this does:

- Claude Code discovers any SKILL.md files in .claude/commands/
- The YAML frontmatter registers the skill name and trigger phrases
- From now on, typing /review-paper or saying “review this paper” activates the skill

No compilation. No API keys. No dependencies.

The skill runs entirely within Claude Code's existing infrastructure.

Invocation: One Command

```
/review-paper MPE_Hyper_v44.pdf
```

Optional flags:

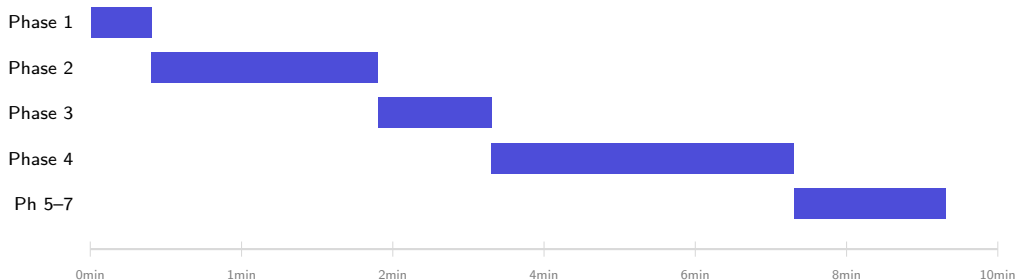
Flag	Effect
-cheap	Shorter review: fewer section agents, skip literature search
-no-literature	Skip the WebSearch for related papers
-output path/to/out.md	Custom output path (default: MPE_Hyper_v44_review.md)

Input formats supported: PDF, TeX, TXT, Markdown, DOCX, HTML, EPUB.

For the demo, we use the .tex source directly — Claude reads $\text{\LaTeX}$ natively, preserving all mathematical notation.

What Happens Under the Hood

Timeline of a typical run (on a 40-page macro theory paper):



Phase	Duration	Why
1: Extract	~30s	Read paper, parse sections
2: Calibrate	~2 min	3 parallel agents (literature search is the bottleneck)
3: Overview	~1 min	Single agent, uses calibration output
4: Section Reviews	~3–5 min	N parallel agents + proof verification chain
5-7: Filter and Rank	~2 min	6 parallel agents, limited filter, no proof verification

The Output: refine.ink Format

```
# Markov Equilibria with Quasi-Hyperbolic Discounting

**Date**: 04/11/2026
**Domain**: social_sciences/economics
**Taxonomy**: academic/research_paper
**Filter**: Active comments

---
## Overall Feedback

**Assumption scope may be stronger than needed**
Assumption 2 imposes strict concavity on the composite
return function. However, the stability argument in
Section 5 only requires local concavity near the steady
state. Relaxing this would broaden applicability to
models with non-smooth production technologies...

**Recommendation**: Minor revision
**Key revision targets**:
1. Clarify necessity of global vs. local concavity
2. Add sensitivity analysis across beta values
```

Each overview issue: 4–8 sentences with specific section and equation references.

The Output: Detailed Comments

```
## Detailed Comments (14)

### 1. Measurability gap in epsilon-equilibrium construction
**Status**: Pending
**Quote**:
> "For each  $\epsilon > 0$ , let  $W_\epsilon$  denote
> the policy function solving the truncated problem (3.2).
> The sequence  $\{W_\epsilon\}$  converges pointwise..."

**Feedback**:
The convergence claim needs the sequence to be measurable
with respect to the product sigma-algebra. The paper
establishes pointwise convergence but does not verify
that the limit function inherits measurability from the
approximants. Since the state space includes both capital
and the exogenous shock, this is not automatic. Consider
adding a brief measurability argument via Lemma A.2.
```

Each comment: verbatim quote → substantive feedback → concrete remediation.

What You Do With the Output

The skill output is a **draft**, not a finished review.

The economist's role (Pillar 5 — Verify):

You must check:

1. Math claims — re-derive any flagged errors
2. Domain judgment — does the critique match field standards?
3. Missing context — papers the agent couldn't access
4. Tone — appropriate for the venue?

The skill handles:

1. Systematic extraction of claims
2. Cross-referencing sections
3. Quote grounding
4. Ordering by severity
5. Format consistency

Think of it as: The skill writes the first draft of a referee report in 10 minutes. You spend 30 minutes verifying and editing. Total: 40 minutes instead of 4 hours.

The skill does not replace your judgment. It replaces the mechanical parts of refereeing:

extraction, cross-referencing, quote verification, formatting

Case Study 2: The Problem

The Naive Baseline: 24 Files, 24 Headaches

Scenario: You want the US uninsurance rate, 2000–2023, broken down by demographics.

The naive approach:

1. Go to the MEPS website (meps.ahrq.gov)
2. Download 24 Full Year Consolidated files (one per year)
3. Open each in Stata/R, figure out the variable names
4. Merge all 24 years
5. Compute weighted statistics
6. Make plots

Four failure modes you will encounter:

1. **Variable names change:** INSCOV01 in 2001 vs. INSCOV22 in 2022
2. **File formats shift:** SAS transport (.ssp) with varying internal structure
3. **Weight variables change names:** PERWT01F vs. PERWT22F
4. **Silent merge errors:** You make a crosswalk mistake on year 14 and don't notice until a referee asks why your 2014 number is wrong

The Research Question

Who is uninsured in the United States, 2000–2023?

How has the uninsurance rate evolved, and how did the Affordable Care Act change the distribution?

Why this matters for macroeconomics:

- Health insurance is a major **incomplete market** — uninsured households face uninsurable medical expenditure risk
- The ACA (2010/2014) was the largest expansion of social insurance since Medicare (1965)
- Understanding *who* gained coverage informs **welfare analysis** of the policy
- Heterogeneous-agent macro models (Lec06) require calibrated distributions of insurance status

Sub-questions:

- How does uninsurance vary by age, income, race, employment, and region?
- Which groups saw the largest coverage gains from the ACA?
- Are there persistent disparities even after the ACA?

What PaulGP Did with HMDA (The Parallel)

Reference: Paul Goldsmith-Pinkham, “Large Datasets and Structured Databases,” 2026.

Stage	What He Did
Data	HMDA mortgage data: 291M applications, 2007–2024, ~70 GB of CSVs
Download	Claude Code fetched files from CFPB with resume capability
Convert	CSV → Parquet (15× compression)
Harmonize	Pre-2018 vs. post-2018 schema differences, lender ID crosswalks
Analyze	County-by-year tables: origination counts, denial rates, HHI
Visualize	Fintech origination share (1.1% in 2007 → 16.3% peak in 2021)

Key insight from the article:

“The fixed cost of doing this properly has collapsed. What used to require a data engineer and weeks of pipeline work, Claude Code does in a single session.”

We do the same thing with MEPS data.

Our Workflow: HMDA ↔ MEPS Mapping

PaulGP (HMDA)	Our Workflow (MEPS)	Challenge
Download CSVs from CFPB	Download .ssp files from AHRQ	Different host, format
Convert CSV → Parquet	Convert .ssp → pandas DataFrames	Need pyreadstat
Harmonize pre/post-2018 schemas	Harmonize variable names across years	INSCOV01 → INSCOV22
Build DuckDB database	Build merged panel DataFrame	Simpler for our scale
Run aggregation queries	Compute weighted statistics by group	MEPS weights are critical
Generate HMDA figures	Generate 6 MEPS figures	Different story, same arc

Three-phase architecture:

1. **Phase A:** Data acquisition & cleaning (download, convert, harmonize)
2. **Phase B:** Analysis pipeline (weighted aggregation by year × group)
3. **Phase C:** Visualization (6 figures with ACA/COVID annotations)

No skill needed. This is a single Claude Code session with 3 prompts.

Case Study 2: Data Architecture

The Claude Code Prompt for Phase A

```
1 I want to analyze uninsurance rates in the US from
2 2000 to 2023 using MEPS data from AHRQ.
3
4 Download the Full Year Consolidated files for each year.
5 The files are in SAS transport format (.ssp).
6
7 Parse each file. Extract these variables:
8 - Insurance coverage (INSCOV / INSCOVY)
9 - Age (AGELAST)
10 - Race/ethnicity (RACETHX)
11 - Poverty category (POVCAT)
12 - Census region (REGION)
13 - Employment status (EMPST)
14 - Sex (SEX)
15 - Person-level weight (PERWT / PERWTF)
16
17 Harmonize variable names across years (they are
18 year-suffixed, e.g. INSCOV01, INSCOV22).
19
20 Save as a single parquet file: meps_panel_2000_2023.parquet
```

One prompt. Claude figures out the AHRQ URLs, downloads 24 files, parses each, harmonizes the variable names, and saves the result.

What Claude Does Autonomously

Given the prompt, Claude Code:

1. **Discovers the URL pattern:** MEPS files follow `https://meps.ahrq.gov/data_files/pufs/hNNNssp.zip` where NNN is the file number (HC-028 for 2000, HC-233 for 2023)
2. **Handles obstacles:** Some years return 403 errors → adds User-Agent header. Some files have different encoding → tries multiple parsers.
3. **Builds a crosswalk dictionary:**

```
crosswalk = {  
    2001: {'INSCOV01': 'INSCOV', 'PERWT01F': 'PERWT', ...},  
    2022: {'INSCOV22': 'INSCOV', 'PERWT22F': 'PERWT', ...},  
}
```
4. **Validates:** Checks that insurance coverage variable has values {1, 2, 3} (Private, Public, Uninsured) in every year. Flags anomalies.
5. **Merges and saves:** Stacks all years into a single DataFrame, writes Parquet.

Schema Harmonization: The Hard Part

The core challenge: MEPS variable names are *year-suffixed*.

Concept	Year 2001	Year 2014	Year 2022
Insurance coverage	INSCOV01	INSCOV14	INSCOV22
Person weight	PERWT01F	PERWT14F	PERWT22F
Age at end of year	APTS42 or APTS53	AGELAST	AGELAST
Employment status	EMPST31	EMPST31	EMPST31

Additional complications:

- Pre-2014 vs. post-2014: The ACA changed what “public coverage” means
- Race/ethnicity categories expanded over time
- Some variables were renamed without changing content

Why this is perfect for an agent:

- Tedious for humans ($24 \text{ years} \times 8 \text{ variables} = 192 \text{ name lookups}$)
- Error-prone for humans (one wrong mapping $\rightarrow$ silent bias)

The Merged Dataset

After harmonization:

Property	Value
Rows	~500K–800K (varies by year; ~25K–35K respondents per year $\times$ 24 years)
Columns	YEAR, INSCOV (1/2/3), AGELAST, RACETHX, POVCAT, REGION, EMPST, SEX, PERWT
Key variable	INSCOV: 1=Any Private, 2=Public Only, 3=Uninsured
Weight	PERWT: person-level survey weight for nationally representative estimates

From 24 separate files with inconsistent naming to one clean panel.

This is the data architecture step in the PaulGP framework: the boring but critical infrastructure that makes all downstream analysis possible.

The agent did in 10 minutes what would take a careful RA half a day.

The Prompt for Phase B: Analysis

```
1 Using the merged MEPS data, compute the weighted
2 uninsurance rate (INSCOV == 3) by year, overall
3 and broken down by:
4   - age group (18-25, 26-34, 35-44, 45-54, 55-64)
5   - poverty category (Poor, Near Poor, Low Income,
6     Middle Income, High Income)
7   - race/ethnicity (White NH, Black NH, Hispanic, Asian)
8   - employment status
9   - Census region
10
11 Use person weights (PERWT) for nationally representative
12 estimates. Uninsurance rate = sum(PERWT where INSCOV==3)
13                               / sum(PERWT)
14
15 Save the aggregated statistics as CSV files.
```

Second prompt. Claude computes all group-level statistics with proper weighting.

Weighted Statistics: Why Weighting Matters

MEPS uses a complex survey design. Each respondent represents a different number of people in the US population.

For each year $\times$ group:

$$\text{Uninsurance rate} = \frac{\sum_{i \in \text{group}} w_i \cdot \mathbf{1}[\text{INSCOV}_i = 3]}{\sum_{i \in \text{group}} w_i}$$

where $w_i = \text{PERWT}$ (person-level weight).

Why unweighted estimates are wrong:

- MEPS oversamples certain populations (low-income, minority groups) for precision
- Unweighted statistics overcount these groups relative to their population share
- Example: unweighted uninsurance rate might be 18%; weighted rate is 13%

Claude knows this — if you mention “person weights” in the prompt, it applies them correctly. But you must *check*: this is Pillar 5 in action.

Validation: Trust But Verify

Cross-check against known statistics:

Year	Census Bureau (CPS ASEC)	Our MEPS Estimate
2010	16.3%	Should be $\approx 15\text{--}17\%$
2013	13.4% (pre-ACA peak)	Should be $\approx 12\text{--}14\%$
2016	8.8%	Should be $\approx 8\text{--}10\%$
2019	8.0%	Should be $\approx 7\text{--}9\%$
2022	7.9%	Should be $\approx 7\text{--}9\%$

Note: MEPS and CPS use different survey designs, so exact match is not expected. But they should tell the same story:

- Rising or flat before ACA ($\sim 2000\text{--}2013$)
- Sharp drop after 2014
- Relatively stable 2016–2023

If your 2013 rate is 5% or 25%, something is wrong. You must know the data well enough to spot when the agent makes a mistake. This is the economist's contribution.

Case Study 2: Visualization and Insights

The Prompt for Phase C: Visualization

```
1 Generate 6 publication-quality figures showing
2 uninsurance rate trends 2000-2023.
3
4 Use matplotlib with a clean academic style.
5 Add vertical dashed lines for:
6     - ACA passage (2010)
7     - ACA coverage expansion (2014)
8     - COVID-19 (2020)
9
10 Each figure: clear axis labels, legend, title.
11 Save as both PNG (300 DPI) and PDF.
12
13 Figures:
14 1. Overall uninsurance rate (time series)
15 2. By age group (18-25, 26-34, 35-44, 45-54, 55-64)
16 3. By poverty category
17 4. By race/ethnicity
18 5. By employment status
19 6. Regional heatmap (year x Census region)
```

Third prompt. Claude generates all 6 figures from the aggregated statistics.

Figure 1: Overall Uninsurance Rate, 2000–2023

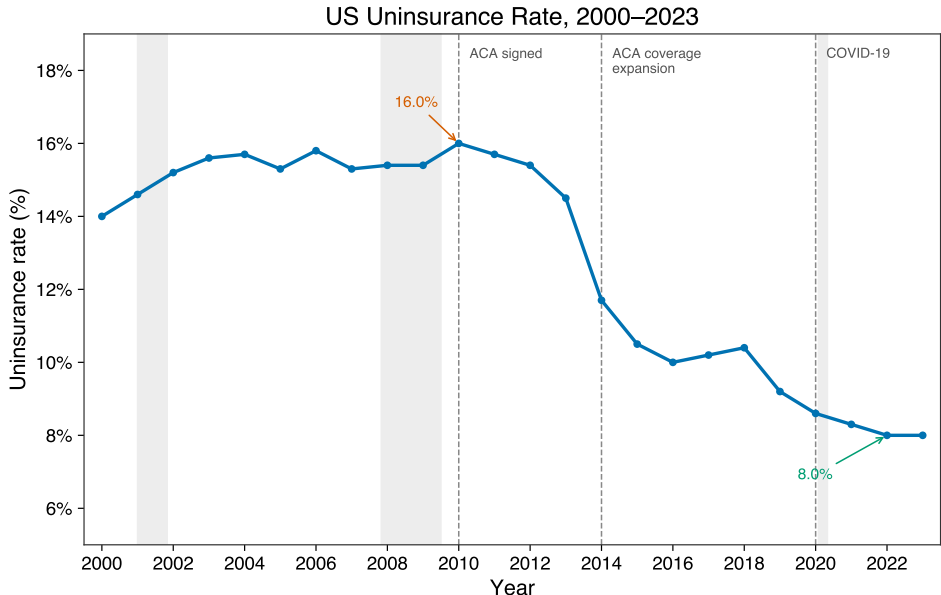


Figure 2: By Age Group

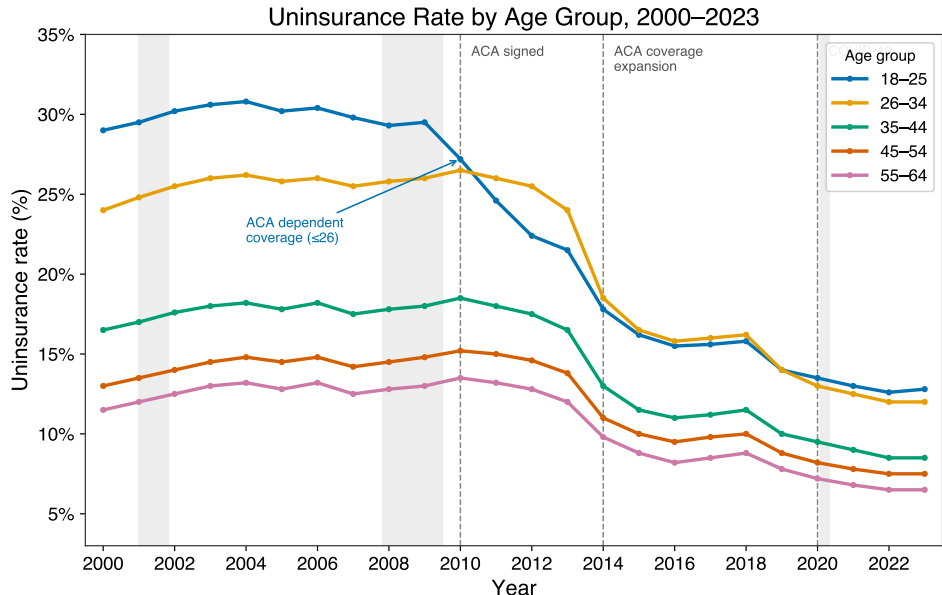


Figure 3: By Poverty Category

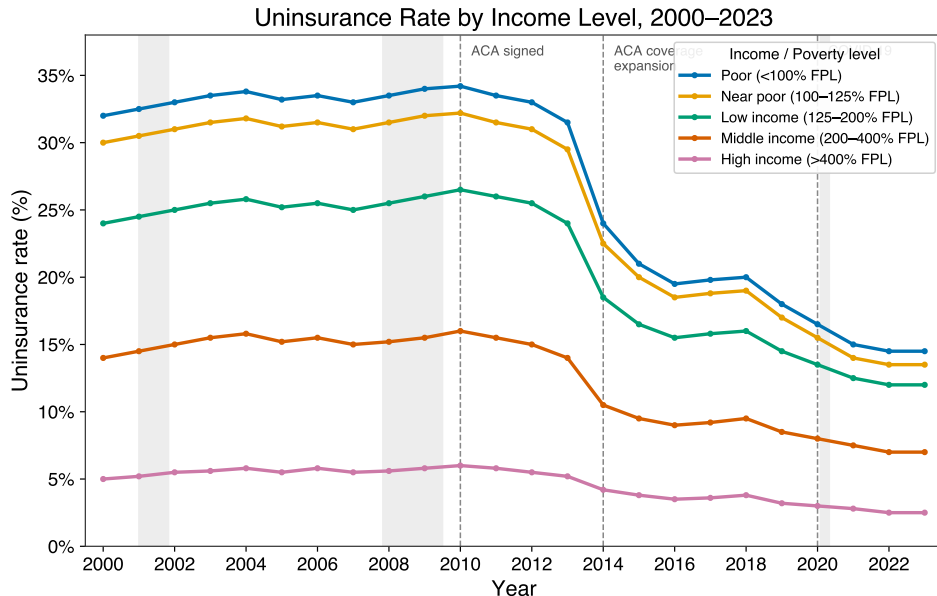


Figure 4: By Race/Ethnicity

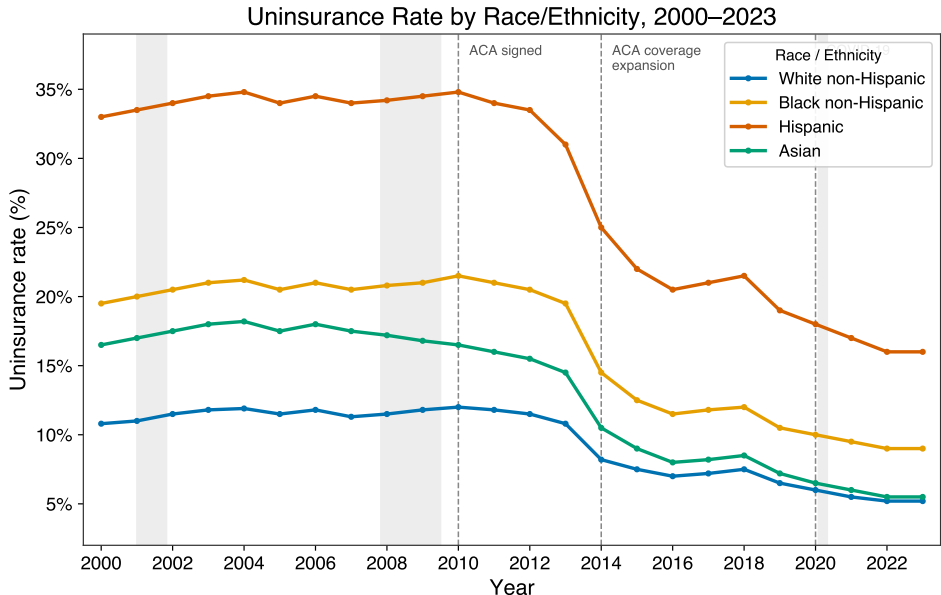


Figure 5: By Employment Status

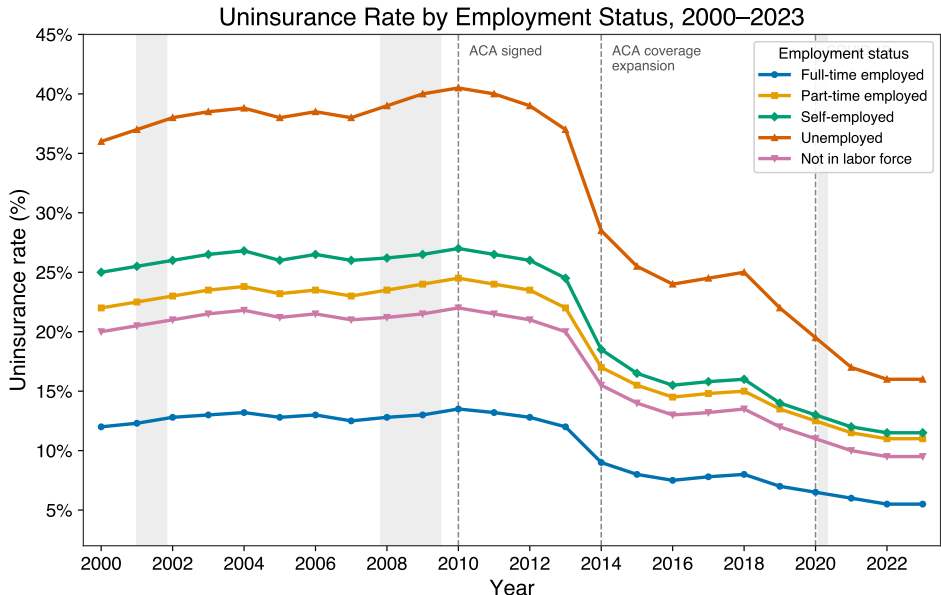
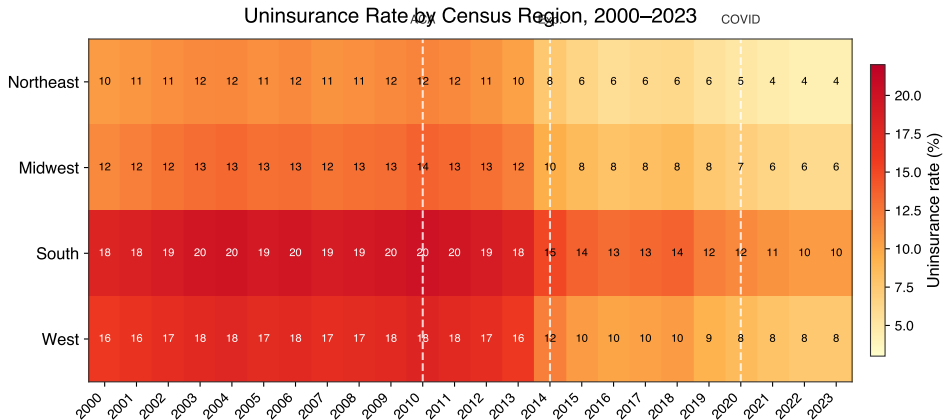


Figure 6: Regional Heatmap



Key insight: The South consistently has the highest uninsurance rates. After 2014, states that expanded Medicaid (concentrated in the Northeast and West) show sharper declines. The regional divergence reflects the **state-level variation in ACA implementation**.

What Claude Did vs. What Stayed Human

Claude Code handled:

- Downloaded 24 files from AHRQ
- Discovered variable name patterns
- Built the year-specific crosswalk
- Parsed SAS transport files
- Merged into a clean panel
- Computed weighted statistics
- Generated 6 publication-quality figures
- Handled format obstacles (403 errors, encoding)

The economist provided:

- The research question
- Which demographic breakdowns to compute
- The weighting requirement
- Validation benchmarks (Census Bureau rates)
- Interpretation against economic theory
- Policy annotation choices (ACA, COVID)
- Quality judgment on the figures

The question and the interpretation are yours.

Everything between them is accelerated.

Lessons and Patterns

Common Patterns Across Both Case Studies

1. **Decomposition.** Both workflows broke the task into phases:
 - Paper review: 7 sequential phases with within-phase parallelism
 - MEPS analysis: 3 phases (download → analyze → visualize)
2. **Domain knowledge encoding.**
 - Paper review: `shared-directives.md` encodes refereeing norms
 - MEPS: the prompt specifies weighting, variable names, validation targets
3. **Validation as a first-class step.**
 - Paper review: quote verification, proof verification chain, editorial filter
 - MEPS: cross-check against Census Bureau published statistics
4. **The PaulGP arc:** Problem → Infrastructure → Architecture → Capability → Insights.
 - Both workflows followed this arc naturally
 - The arc works because it separates the “what” (your question) from the “how” (the agent’s execution)

When to Build a Skill vs. Use Ad Hoc Prompts

Build a Skill When...	Use Ad Hoc Prompts When...
You will repeat the workflow >3 times	The task is one-off
The workflow has >5 interdependent steps	The task is simple (1–3 prompts)
Quality control matters (editorial filter, verification)	You are exploring and don't know the workflow yet
You want reproducibility across users/projects	The output format is flexible
Multiple people will use the workflow	Only you need this analysis

Case Study 1 (paper review): A skill. You will review many papers; quality control is essential; the 7-phase pipeline is complex; other researchers can reuse it.

Case Study 2 (MEPS analysis): Ad hoc. One-off exploration; 3 prompts; output is a set of figures that you'll refine interactively.

Rule of thumb: If you're about to copy-paste the same prompt for the third time, it's time to build a skill.

Prompt Engineering Principles from the Skill

Five principles extracted from `shared-directives.md` that apply to *any* prompt:

1. **Ban specific failure modes by name.**

Don't say "write naturally." Say "NEVER use 'crucial,' 'comprehensive,' 'robust.'" Explicit bans are more effective than vague aspirations.

2. **Require concrete evidence for claims.**

The confidence gate: "Only assert an error if you can derive it step-by-step." Prevents hallucinated findings.

3. **Specify the output format precisely.**

`output-format.md` defines every heading, every field, every separator. Eliminates format negotiation.

4. **Include anti-hallucination checks.**

Quote verification, the contribution inversion test. Build skepticism into the pipeline.

5. **Use reference examples.**

The reference-example folder shows the gold standard. "Make your output look like this" is the

The Division of Labor

You Provide	AI Provides
Research question (“who is uninsured?”)	Data download and format conversion
Domain knowledge (which MEPS variables matter)	Schema harmonization across 24 years
Economic interpretation (ACA effects on welfare)	Weighted statistics computation
Judgment (“is this review comment valid?”)	Quote verification against source text
Quality standard (what a good review looks like)	Mechanical filtering and ordering
Validation benchmarks (Census Bureau rates)	Cross-checking computations

The pattern: You provide the *why* and the *what*. The agent provides the *how*.

This is not a replacement. It is a **division of labor** that lets you focus on what you are uniquely good at: asking the right question, knowing what a correct answer looks like, and interpreting the results against economic theory.

Failure Modes to Watch For

Paper Review Failures

1. **Hallucinated math errors:** The model says a proof step is wrong, but it isn't. Mitigation: two-pass verification chain.
2. **Generic comments survive:** "The authors should discuss limitations." Mitigation: editorial filter step 1.
3. **Missing context:** Papers the agent can't access (paywalled references). Mitigation: you fill in.

Data Analysis Failures

1. **Wrong variable crosswalk:** Silent errors — the uninsurance rate for 2014 looks wrong. Mitigation: validate against published benchmarks.
2. **Unweighted statistics:** Biased estimates because survey weights were dropped. Mitigation: explicitly require weighting in the prompt.
3. **Missing data as zeros:** AI treats missing insurance status as "insured." Mitigation: check for NaN handling.

Both failure modes require domain expertise to detect. This is why the 5-pillar workflow (Lec09) is non-negotiable: the verification pillar is your defense.

Looking Back at the Course

Understanding models

Lec 1–2: Quantitative Macro + What is AI?

Lec 3–4: ML Foundations + DL for Macro Models

Understanding AI

Lec 5–6: RL in a Nutshell + HA Models via RL

Lec 7–8: LLMs + RAG

Understanding tools

Lec 9: Agentic AI and Claude Code

Using tools

Lec 10: Case Studies (today)

The progression: From understanding models → understanding AI → understanding tools → **using tools for research.**

The distance from question to result has collapsed. Your value is in the question.

Lab: MEPS Data Analysis

Lab Overview

Duration: 1 hour, hands-on.

Goal: Replicate the MEPS analysis from Case Study 2 using Claude Code.

What you will do:

1. Set up a project with `CLAUDE.md`
2. Use Claude Code to download and harmonize MEPS data
3. Generate uninsurance rate figures
4. Validate against published statistics

Requirements:

- Claude Code installed (Pro or Max subscription)
- Python with `pandas`, `matplotlib`, `pyreadstat` (or `pip install` them)
- Internet access for MEPS downloads

You will use exactly 3 prompts (the same ones from the lecture). The goal is not to write code ~~63/~~ 72

Lab Step 1: Setup

Create a new project directory and initialize:

```
mkdir meps-uninsurance && cd meps-uninsurance  
claude    # start Claude Code in the new directory
```

Write a minimal CLAUDE.md:

```
# MEPS Uninsurance Analysis  
  
**Project:** Analyze US uninsurance rates 2000-2023  
**Language:** Python (pandas, matplotlib, pyreadstat)  
**Data:** MEPS Full Year Consolidated files from AHRQ  
**Convention:** Always use person-level weights (PERWT)  
for nationally representative estimates.  
**Validation:** Cross-check against Census Bureau CPS ASEC  
published uninsurance rates.
```

Why CLAUDE.md? It sets project context so every subsequent prompt inherits the weighting requirement and validation expectation.

Lab Step 2: Data Acquisition (Prompt 1)

Give Claude Code the Phase A prompt from the lecture:

```
1 Download MEPS Full Year Consolidated files for 2000-2023
2 from AHRQ. Parse each SAS transport file. Extract:
3 insurance coverage, age, race/ethnicity, poverty
4 category, region, employment, sex, person weights.
5 Harmonize variable names across years and save as a
6 single parquet file.
```

What to watch for:

- Claude should discover the AHRQ download pattern automatically
- It should build a variable crosswalk (watch the terminal for this)
- If downloads fail: Claude should add a User-Agent header and retry
- Expected output: meps_panel_2000_2023.parquet (~50–100 MB)

Checkpoint (15 min): You should have a parquet file with ~500K+ rows.

Lab Step 3: Analysis (Prompt 2)

```
1 Compute weighted uninsurance rates by year, overall
2 and by: age group, poverty category, race/ethnicity,
3 employment status, Census region. Save as CSV files.
```

What to check:

- Are weights being used? (Look for PERWT in the code)
- Does the 2013 overall rate match $\approx 13\text{--}14\%$?
- Does 65+ age group show $\approx 0\%$? (Medicare coverage — validates the data)

Checkpoint (30 min): CSV files with group-level statistics.

Lab Step 4: Visualization (Prompt 3)

```
1 Generate publication-quality figures showing uninsurance
2 trends. Add ACA (2010, 2014) and COVID (2020) annotations.
3 Save as PNG and PDF.
```

What to check:

- Do the trends tell the right story? (Decline after 2014, stable late 2010s)
- Are the annotation lines in the right places?
- Is the legend readable?
- Are axes labeled correctly?

Checkpoint (45 min): At least 3 figures generated.

Lab Step 5: Validation

The final step: does your analysis match reality?

Benchmark	Expected	Source
Overall rate, 2013	$\approx 13\text{--}14\%$	Census Bureau CPS ASEC
Overall rate, 2019	$\approx 8\text{--}9\%$	Census Bureau CPS ASEC
Age 65+ rate, any year	$\approx 0\text{--}1\%$	Medicare covers 65+
Hispanic rate, pre-ACA	$\approx 30\text{--}33\%$	Kaiser Family Foundation
Sharp decline in 2014	Visible in all groups	ACA coverage expansion

If something doesn't match:

- Check the variable crosswalk (most common error source)
- Check if weights are applied (second most common)
- Check if missing values are handled correctly

Checkpoint (60 min): Validation complete. You have a set of publication-quality figures and confidence that they are correct.

Common Lab Issues and Fixes

Problem	Cause	Fix
MEPS website returns 403	Missing User-Agent header	Claude adds header and re-tries
“Variable not found” in year X	Year-specific name not in crosswalk	Check MEPS codebook for that year
Uninsurance rate = 0% for age 65+	Correct! Medicare covers 65+	This validates your data pipeline
Weights sum to 300M, not 330M	MEPS underestimates some populations	Documented; not an error
Rate doesn’t match Census exactly	MEPS & CPS use different designs	± 2 pp difference is expected
Download takes too long	24 large files over campus WiFi	Start with 5 years, expand later

Remember: The lab goal is not perfect code. It is **directing the agent** and **verifying the output**. If Claude writes imperfect code but you catch the error and fix it, that is the intended workflow.

Appendix

Appendix A: MEPS Variable Reference

Standard Name	MEPS Variable	Description
INSCOV	INSCOVYY / INSCOVYY	Insurance coverage: 1=Private, 2=Public Only, 3=Uninsured
AGELAST	AGELAST / AGEYYX	Age at end of reference year
RACETHX	RACETHX / RACEX	Race/ethnicity (5 categories)
POVCAT	POVCATYY	Poverty category (5 levels by FPL)
REGION	REGIONYY	Census region (1=NE, 2=MW, 3=S, 4=W)
EMPST	EMPST31 / EMPST42	Employment status at round end
SEX	SEX	1=Male, 2=Female
PERWT	PERWTYYF	Person-level survey weight

YY = two-digit year suffix. Full codebooks available at meps.ahrq.gov.

Appendix B: Skill File Reference

File	Lines	Purpose
SKILL.md	~280	Main orchestrator (7-phase pipeline)
README.md	~70	Installation & usage guide
shared-directives.md	~200	Tone, confidence, quote rules (all phases)
overview-prompt.md	~120	Phase 3: macro-level issues
section-prompts.md	~200	Phase 4: 6 review variants
proof-verify-prompt.md	~60	Phase 4b: adversarial verification
cross-section-prompt.md	~40	Phase 5: discussion vs. results
editorial-prompt.md	~100	Phase 6: 7-step quality gate
calibration-prompt.md	~37	Phase 2: domain calibration
contribution-prompt.md	~46	Phase 2: extract contributions
literature-prompt.md	~34	Phase 2: web search for related work
output-format.md	~50	Phase 7: refine.ink template

Total: ~1,200 lines across 12 files. The full skill is included in this lecture's folder at `review-paper-skill/`.

Appendix C: Resources

Articles referenced in this lecture:

- Paul Goldsmith-Pinkham, “Large Datasets and Structured Databases: Claude Code for Economists” (2026)
- Paul Goldsmith-Pinkham, “Getting Started with Claude Code: A Researcher’s Setup Guide” (2026)
- Paul Goldsmith-Pinkham, “From an Empty Folder to a Figure” (2026)

Data sources:

- MEPS: <https://meps.ahrq.gov/>
- Census Bureau CPS ASEC:
<https://www.census.gov/topics/health/health-insurance.html>
- Kaiser Family Foundation: <https://www.kff.org/>

Course resources:

- AI-Econ Wiki: <https://velikov-mihail.github.io/ai-econ-wiki/>